操作系统原理实验

实验报告

实验一
进程控制与描述

基于 Win32 API 的进程创建、观察与终止

项目	信息
班级	药学计算机 2401
学号	302024514086
姓名	徐茄伟
实验形式	个人实验
验证日期	2026 年 10 月 19 日

实验概要

编写兼容传统 Win32 编程方式的控制台程序，通过 CreateProcessA 创建可控子进程，显示父子进程标识，使用 TerminateProcess 发出终止请求，并用等待对象和退出码确认执行结果。同时验证子进程主动 ExitProcess、创建失败及提前退出情形。

实现	验证结果
交互功能	创建成功、终止成功和创建失败均有真实提示框
自动检查	4 项检查全部通过；编译零警告
随附材料	完整源码、Word/PDF 报告、原始日志与窗口截图

验证范围：实际证据来自 MinGW-w64 编译和 Wine 10.0 运行；未在 Windows 2000/XP + Visual C++ 6.0 原生环境实测。报告保留原课件环境要求，并给出相应复核步骤。

一、实验目的与实验内容

1.1 实验目的

· 掌握 Windows 下创建进程与终止进程的基本方法，理解进程是程序的一次执行活动。

· 认识进程对象、主线程、进程/线程标识及句柄的区别，理解父子进程可以并发执行。

· 通过可观察的输出、返回值和退出码分析进程状态变化，形成检查错误与释放资源的习惯。

1.2 实验内容

按照课件第 12、15 页，编写程序创建并终止“当前创建的进程”，对操作成败给出提示，观察并分析结果，提交实验报告和源代码。本实现默认启动同一程序的子进程模式，避免不同系统缺少写字板而影响复现；另提供 --app 参数启动外部可执行文件。

1.3 实验环境及验证边界

项目	课件要求 / 实际处理
指定环境	Windows 2000/XP，Visual C++ 6.0
实际编译	Debian 13 x86_64；MinGW-w64 GCC 14-posix；C90 严格检查
实际运行	Wine 10.0 + Xvfb，运行真正的 Windows PE 程序
兼容设计	单个 ANSI C 源文件；使用传统 Win32 API；无 C99 声明及 C++11 特性
原生复核	提供 VC6 工程建立步骤和 cl 编译脚本；该环境尚未实测

1.4 相关概念

程序是磁盘上的静态代码与数据；进程包含独立虚拟地址空间、资源和执行上下文。Windows 以线程作为调度实体：CreateProcess 同时创建进程及其主线程。相同 EXE 可对应多个 PID，父子进程不会因执行同一份文件就共享普通全局变量。

PID/TID 是标识值，不是具有访问权限的句柄。PROCESS_INFORMATION 返回的 hProcess 用于等待、查询退出码及终止，hThread 指向主线程。关闭句柄只释放本进程对对象的引用，不等于结束目标进程。

观察时，子进程周期性 Sleep，主线程在运行、就绪和等待之间转换；父进程显示模态提示框时子进程仍可运行，体现独立调度。仅凭窗口仍存在或 CPU 占用为零，都不能断定进程是否已经终止。

二、总体设计与关键 API

2.1 程序组织

运行方式	作用
无参数	创建自身 --child-visible 模式的新控制台；观察后点击确定终止
--self-test	无需逐个确认提示框，执行 4 项可判定的自动检查
--app 完整路径	创建指定外部程序；仍只操作本次返回的进程句柄
内部子进程模式	--child-natural 主动 ExitProcess(23)；--child-wait 等待终止

2.2 CreateProcessA 参数设置

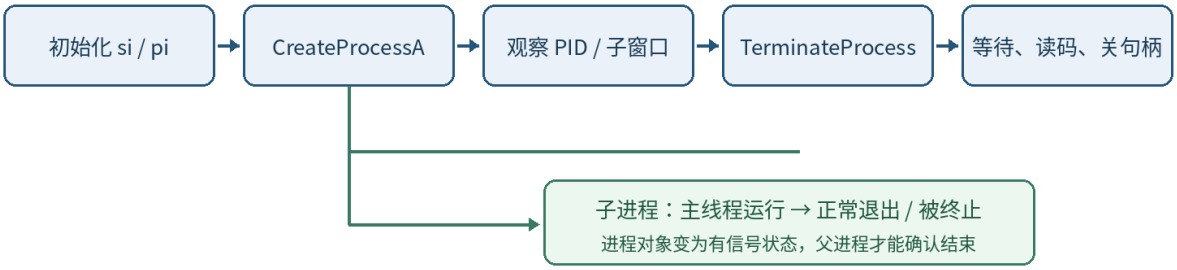
参数	本实验设置与含义
lpApplicationName	显式给出完整 EXE 路径，避免可执行文件名称解析歧义。
lpCommandLine	可写字符数组，形式为“加引号的 EXE 路径 + 模式参数”。
lpProcessAttributes	NULL：返回的进程句柄使用默认安全属性且不可继承。
lpThreadAttributes	NULL：返回的线程句柄使用默认安全属性且不可继承。
bInheritHandles	FALSE：不让子进程继承父进程的可继承句柄。
dwCreationFlags	交互演示 CREATE_NEW_CONSOLE；自动检查使用 0。
lpEnvironment	NULL：采用调用进程的环境。
lpCurrentDirectory	NULL：采用调用进程的当前目录。
lpStartupInfo	指向清零后的 STARTUPINFOA，必须设置 si.cb。
lpProcessInformation	指向 PROCESS_INFORMATION，成功时获得句柄、PID 和 TID。

2.3 终止和确认

ExitProcess(23) 由子进程主动调用，结束调用者自身。TerminateProcess(pi.hProcess, 37) 由父进程对目标进程发出强制终止请求，句柄必须包含 PROCESS_TERMINATE 权限。本实验使用 CreateProcess 返回的句柄，无需再次按 PID 打开进程。

TerminateProcess 返回成功后仍需 WaitForSingleObject 等待进程对象变为有信号状态，再使用 GetExitCodeProcess 获取最终结果。先等待再读退出码，可以避免把 STILL_ACTIVE（259）与最终状态混淆；程序未将 259 用作退出码。

三、程序流程与关键实现



创建失败：立即记录 GetLastError 并提示；提前退出：直接收集退出状态。

程序控制流程（设计图；完整实现在源代码/ProcessLab.c）

3.1 初始化与创建

```
ZeroMemory(&si, sizeof(si));
ZeroMemory(pi, sizeof(*pi));
si.cb = sizeof(si);
/* commandLine is a writable, quoted character buffer. */
CreateProcessA(application, commandLine, NULL, NULL, FALSE,
               flags, NULL, NULL, &si, pi);
```

源码先用 GetModuleFileNameA 获取自身路径，检查缓冲区长度并构造带引号的命令行。创建成功后显示父 PID、子 PID、主线程 TID；主线程句柄不再使用时立即关闭，进程句柄保留到控制结束。

3.2 检查、终止、等待与查询

```
/* Full error handling is in StopChild and WaitAndRead. */
result = WaitForSingleObject(pi->hProcess, 0);
if (result == WAIT_TIMEOUT) {
    TerminateProcess(pi->hProcess, FORCED_EXIT_CODE);
    WaitForSingleObject(pi->hProcess, WAIT_LIMIT_MS);
    GetExitCodeProcess(pi->hProcess, &exitCode);
}
CloseChild(pi);
```

上段仅为流程节选。完整代码逐一检查 API 返回值，立即保存 GetLastError，通过 FormatMessageA 解释错误；终止后仅在等待成功且退出码符合预期时给出成功提示。等待超时会报告“未确认终止”，不会误报成功。

3.3 边界与资源处理

如果用户提前关闭子进程，StopChild 检测到进程对象已变为有信号状态，就直接收集退出码并提示“已经退出”。若进程在检查后、终止前退出，则再次确认其状态，避免把正常竞争情形当成无法解释的错误。

父进程仅关闭自己取得的句柄；对创建失败的路径不使用无效句柄。程序结束前关闭日志文件。所有控制行为限定为本次 CreateProcess 返回的对象，不按程序名批量终止其他实例。

四、实验步骤与复现方法

4.1 Visual C++ 6.0 工程建立

① File → New → Projects → Win32 Console Application，项目名设为 ProcessLab，选择 An empty project。

② 通过 Project → Add To Project → Files 加入 ProcessLab.c；不要选择依赖 stdafx.h 的预编译头模式。

③ 在 Project → Settings → Link 的 Object/library modules 中加入 user32.lib，保留系统默认库。

④ 按 F7 编译，Ctrl+F5 运行。若使用命令行，先运行 VC98/Bin/VCVARS32.BAT，再执行附带 build_vc6.bat。

4.2 创建、观察并终止进程

① 无参数运行 ProcessLab.exe。程序打开子进程控制台，父进程弹出创建成功提示框。

② 记录父 PID、子 PID、主线程 TID。子窗口按约 1 秒间隔输出 PID 和 tick，说明子进程仍在执行。

③ 原生 Windows 上可打开任务管理器，在“进程”或“详细信息”页按 PID 定位本次创建的实例；父子进程同名时必须用 PID 区分。

④ 点击父进程提示框的“确定”。子进程被终止，程序等待确认后显示成功提示及退出码 37；查看 process_lab.log。

任务管理器观察是原生复核步骤，不属于本次云端实际截图。每次父进程启动会覆盖当前目录的 process_lab.log，保留结果时应先复制日志。

4.3 自动检查与失败路径

```
ProcessLab.exe --self-test
ProcessLab.exe --app "C:\Windows\System32\notepad.exe"
ProcessLab.exe --app "C:\NoSuchLabFolder\missing.exe"
```

第一条自动检查 4 条路径，成功时返回 0。第二条为可选外部应用演示，请只使用没有未保存内容的测试实例；部分新版记事本会转交给其他进程，优先使用默认受控子进程。第三条用于观察创建失败提示。

4.4 本次实际验证命令

```
x86_64-w64-mingw32-gcc-posix -std=c90 -pedantic \
  -Wall -Wextra -Werror -O2 ProcessLab.c \
  -o ProcessLab.exe -luser32
wine64 ProcessLab.exe --self-test
```

严格交叉编译退出码为 0，编译输出为空（无警告、无错误）。测试用可执行文件只用于当前验证，不放入提交包，符合课件“仅提交源代码，不提交目标文件和可执行文件”的要求。

五、运行结果：自动检查与失败提示

最终提交源码重新编译后，自动检查返回 0，汇总为 4/4 PASS。以下为实际结果，不是预期输出模板。完整逐行日志见“验证记录/自动测试_原始日志.txt”。

检查项	实际观察	判定与原因
1. 主动结束	子 PID=64；最终退出码 23	通过：子进程执行 ExitProcess(23)，父进程等待并读到正确退出码。
2. 强制终止	子 PID=72；终止前等待结果 258；最终退出码 37	通过：258 为 WAIT_TIMEOUT，说明尚未结束；终止后进程对象有信号且退出码匹配。
3. 创建失败	不存在的 EXE 返回错误码 2；未创建有效子进程	通过：这是测试所要求的失败结果，不代表正常创建功能失败。
4. 提前退出	子 PID=80；退出码 23；显示 ALREADY EXITED	通过：不重复强制结束已结束对象，仍收集退出状态并释放句柄。

5.1 原始日志节选

```
[05:52:26] TEST 1 PASS
[05:52:26] TEST 2 PASS
[05:52:26] TEST 3 PASS (expected creation failure)
[05:52:26] TEST 4 PASS
[05:52:26] SELF-TEST RESULT: 4/4 PASS
```

本次自动检查父 PID 为 32。PID/TID 与时刻均是运行时信息，重新运行不必得到相同数值；判定依据是 API 行为、等待结果和退出码。原始日志采用 UTC 时钟。

5.2 交互创建失败检查

运行 --app "C:\NoSuchLabFolder\missing.exe" 后，实际出现错误提示框，GetLastError 为 3（路径不存在），父进程返回 1。它与自动检查的错误码 2（文件不存在）对应不同失败前提，两者均符合系统错误语义。



图 4 创建失败真实提示（Wine 10.0；错误码 3）

六、运行结果：交互观察与分析

以下均为本程序在 Wine 兼容层中的真实窗口捕获。弹窗通过窗口确认消息继续执行；未替换或模拟 CreateProcess、TerminateProcess 的返回值。

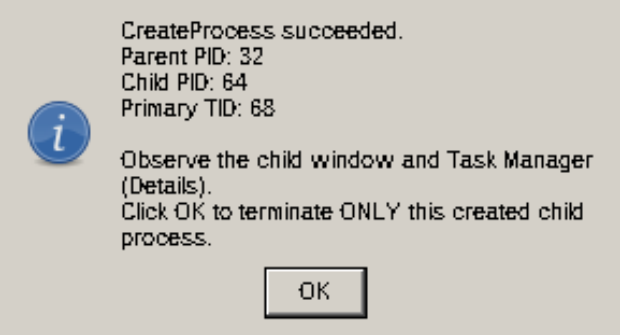


图 1 创建成功：父 PID=32、子 PID=64、主线程 TID=68

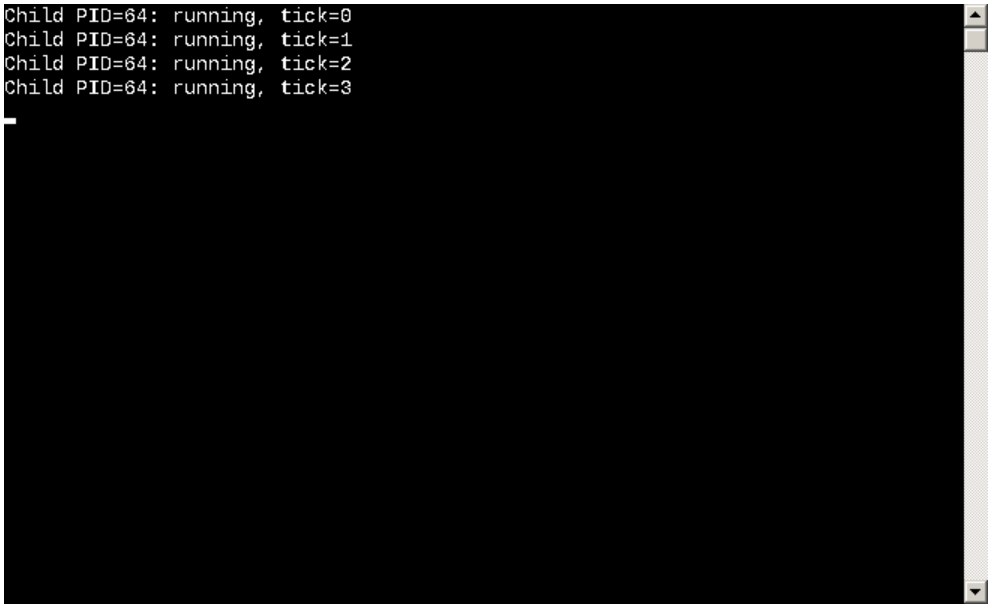


图 2 子进程独立运行：周期输出 PID 和 tick



图 3 终止请求成功，等待后确认退出码 37

结果分析：父进程弹窗等待期间，子窗口输出与之独立，表明两者可并发推进；点击确定后，父进程记录终止请求，再记录 EXIT CONFIRMED，最终释放句柄。37 是程序自行规定的演示退出码，不是系统错误码。

这些现象说明了应用可观察到的进程控制行为；截图不直接证明内核调度细节，也不能替代原生 Windows 任务管理器记录。完整交互时序见“交互演示_原始日志.txt”。

七、问题分析与解决方法

7.1 测试中实际观察到的问题

问题 / 现象	原因与处理
无效路径导致 CreateProcess 失败	区分错误码 2（文件不存在）和 3（路径不存在）；立即保存 GetLastError，用 FormatMessageA 显示系统说明，不再读取未初始化句柄。
目标可能在父进程操作前退出	自动检查确认 ALREADY EXITED 路径可正常处理。先检查进程对象是否有信号；已经结束时直接获取退出码，避免误报终止失败。
课件指定环境与云端平台不同	使用 MinGW 生成 Windows PE 并以 Wine 验证 API 行为；保留 VC6 语法和构建步骤。报告明确环境边界，不把兼容层证据写成原生实测。

7.2 实现中采取的预防性处理

（1）结构体初始化：STARTUPINFOA 先清零，再设置 cb；PROCESS_INFORMATION 先清零，避免失败路径使用垃圾句柄。

（2）命令行和路径：显式设置 EXE 完整路径，命令行路径加引号，且传入可写数组；检查截断并报告长度错误，避免含空格路径被错误拆分。A 版接口对 ANSI 代码页有边界，默认仅要求当前系统可表示的路径。

（3）异步终止：TerminateProcess 成功表示请求成功；程序继续等待最多 5 秒并验证退出码，超时单独报告，不把请求接受等同于结束完成。

（4）句柄和文件生命周期：主线程句柄用完即关，进程句柄在查询结束后关闭；日志写入失败只发警告，实验控制仍可执行。关闭句柄不是终止进程的替代操作。

7.3 对课件表述的必要澄清

课件第 10 页中“uExitCode=0 表示当前进程”的说法不适用于 TerminateProcess：hProcess 指定目标，uExitCode 只是给目标使用的退出码；传 0 表示退出码为 0。ExitProcess 才是直接结束调用者自身的接口。

TerminateProcess 是强制终止手段，不保证执行目标程序的清理逻辑和 DLL 正常退出通知，可能丢失未保存数据。真实软件宜优先协作退出；本实验为了观察强制控制，只终止自己创建的测试进程。正常返回 main、最后线程退出或未处理异常等也可能结束进程，不能把所有结束情形都简化为应用显式调用这两个 API。

八、思考题与实验总结

8.1 系统是怎样创建进程的？

应用调用 CreateProcess 后，系统解析并检查映像与访问权限，创建进程对象、虚拟地址空间及相关安全/资源环境，准备进程参数和句柄表，映射可执行映像，并创建主线程及其初始上下文。未设置 CREATE_SUSPENDED 时，主线程可被调度。Windows 还需完成用户态加载初始化；API 返回成功不保证应用窗口已出现或全部初始化已成功，后续仍要观察运行状态。

8.2 可执行文件加载时进行了哪些处理？

系统按 PE 头检查映像及目标体系结构，将代码、数据等节映射到虚拟地址空间并设置适当保护，必要时处理基址重定位；加载器解析依赖 DLL 和导入符号，完成 DLL 与 TLS 等初始化，准备用户态执行环境。程序的磁盘文件并非简单“复制到物理内存”：映像可采用文件映射和按需调页，实际页面由内存管理机制按访问情况提供。

8.3 首次运行新进程时，其入口在哪里？

主线程先经过系统启动与加载器初始化路径，再进入 PE 头 AddressOfEntryPoint 指示的入口（相对虚拟地址需结合映像基址）。对普通 VC 控制台程序，该入口通常是 CRT 启动代码，例如 mainCRTStartup；CRT 完成运行库等初始化后才调用用户 main。GUI 程序通常经 WinMainCRTStartup 再调用 WinMain。因此不能把用户 main 等同于操作系统最早执行的指令；最终入口以实际 PE 及链接配置为准。

8.4 实验总结

本实验实现了“创建—观察—终止—确认—释放”的完整控制链。四项检查和真实提示框验证了正常、强制及异常分支：进程标识用于定位，对象句柄用于受控访问，等待对象用于确认终止。完整源码与验证证据随报告提交；尚未覆盖原生 VC6 编译、Win2000/XP 任务管理器及所有外部应用的转交行为。

参考资料

［1］课程课件《实验一 进程控制与描述》，第 2—15 页。

［2］Microsoft Learn：CreateProcessA；创建参数与返回信息。https://learn.microsoft.com/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa

［3］Microsoft Learn：TerminateProcess、ExitProcess；终止语义与清理限制。https://learn.microsoft.com/windows/win32/api/processthreadsapi/nf-processthreadsapi-terminateprocess

［4］Microsoft Learn：WaitForSingleObject、GetExitCodeProcess；等待和退出状态。https://learn.microsoft.com/windows/win32/api/synchapi/nf-synchapi-waitforsingleobject

［5］Microsoft Learn：/ENTRY (Entry-Point Symbol)；CRT 与用户入口。https://learn.microsoft.com/cpp/build/reference/entry-entry-point-symbol